

Software Engineering Project

Jan 2026 Term - Team 051

Milestone 4 (Sprint 2)



Submitted by:

Kaushik Gaur (23f1000687)

Shriyanshi Parashari (22f3001012)

Aadil Iqbal (23f1001130)

Nachiket Gore (23f1000542)

1. Changes made based on user feedback

This document details the changes implemented in Sprint 2 based on user feedback collected at the end of Sprint 1 (Milestone 3). During the Sprint 1 review session, the business owner identified two areas for improvement in the Manager Dashboard — the Analytics section and the Settings section. This document covers each feedback point with the original request, the exact changes made in the database, backend, and frontend, before/after comparisons, and visual proof via application screenshots.

Two feedback points were received:

1. Analytics — Add a dedicated 'Items Sold' summary card to the Analytics overview.
2. Settings — Add a complete Billing control section with restaurant identity fields, GST rate, terms & conditions, payment QR code, custom dish profit margin, selling price, and kitchen tracking.

2. Feedback #1 — Analytics: Items Sold Card

Section: Manager Dashboard → Analytics

Priority: Medium

Original Feedback: Add a dedicated "Items Sold" summary card on the Analytics page at the same level as Revenue, Orders Closed, and Average Order Value cards.

What Was Already Present (Sprint 1)

The Items Sold card was already implemented in Sprint 1 after taking the user feedback. The backend analytics API (GET /api/manager/analytics) computes and returns items_sold in the summary object by summing OrderItem.quantity across all closed orders in the selected date range. The frontend Analytics.js component displays it as the 4th card in the summary grid with a red (#E63946) left border accent and the sub-label "Across closed orders".

Verification in Sprint 2

During Sprint 2, the Items Sold card was verified to be working correctly across all date range selections. No code changes were required for this feedback point — the implementation already matched the user's request. The following was confirmed:

- The card displays at the same level as Revenue, Orders Closed, and Average Order Value cards in a 4-column grid layout.
- The card value updates correctly when the manager changes the date range selector (Last 7 Days / Last 30 Days / Last 90 Days / All Time).
- The card shows "0" clearly when no items are sold in the selected range — it is not hidden.

- The card has a distinct red accent border and clear "ITEMS SOLD" uppercase label matching the style of other summary cards.
- The sub-label reads "Across closed orders" providing context for the number shown.

Visual Proof

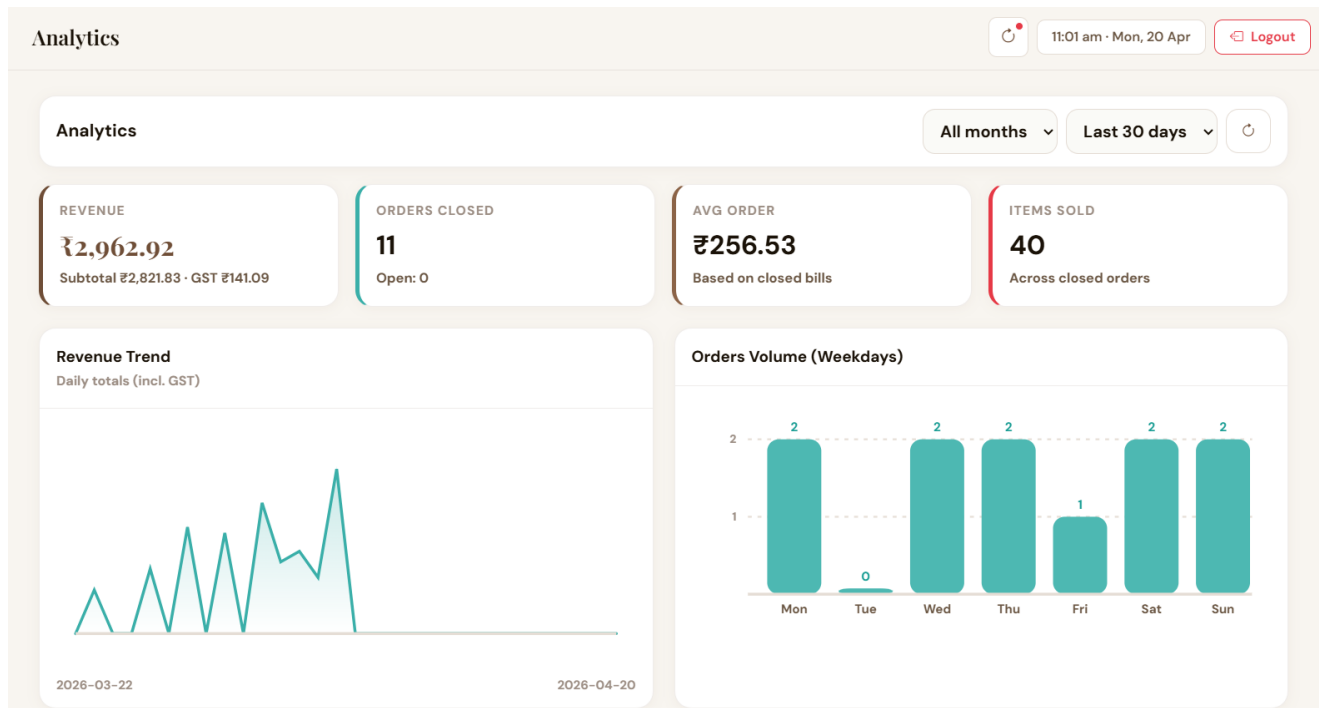


Image 1 - Analytics page showing all 4 summary cards

STATUS: No code changes needed — already implemented in Sprint 1 after taking the user feedback.

The Items Sold card was present and functional in both the backend API and the frontend. Sprint 2 verified its correctness across all date range selections.

3. Feedback #2 — Settings: Billing Control Section

Section: Manager Dashboard → Settings

Priority: High

Original Feedback: Add a dedicated Billing section inside Settings with full control over bill appearance — restaurant name, GSTIN, address, phone toggles, GST rate, terms & conditions, payment QR, custom dish profit margin, selling price field, and kitchen status/delivered_at tracking.

3.1 Database Changes Made

Manager Table — New Columns

The following columns were added to the managers table to store billing configuration:

Column	Type	Purpose
restaurant_name	VARCHAR(160)	Restaurant name displayed on bills
address	VARCHAR(255)	Restaurant address on bills
gstin	VARCHAR(32)	GSTIN number on bills
phone_number	VARCHAR(32)	Contact number on bills
gst_rate	FLOAT	Configurable GST % (e.g. 5, 12, 18)
bill_terms	TEXT	Custom terms & conditions text
qr_image_path	VARCHAR(255)	Path to uploaded payment QR image
bill_show_restaurant_name	BOOLEAN	Toggle: show/hide name on bill
bill_show_address	BOOLEAN	Toggle: show/hide address on bill
bill_show_gstin	BOOLEAN	Toggle: show/hide GSTIN on bill
bill_show_phone	BOOLEAN	Toggle: show/hide phone on bill
custom_dish_profit_margin	FLOAT	Profit margin % for AI custom dishes

Dishes Table — New Column

Column	Type	Purpose
selling_price	FLOAT	Customer-facing price (overrides base_price if set)

Order Items Table — New Columns

Column	Type	Purpose
kitchen_status	VARCHAR(20)	Item state: queued → done → delivered
delivered_at	DATETIME	Timestamp when waiter delivered the item

3.2 Backend API Changes

Settings API — GET / PUT /api/manager/settings

Before (Sprint 1): The settings endpoint only returned basic restaurant info — name and email. There were no fields for billing controls, GST rate, terms, or QR path.

After (Sprint 2): The settings endpoint now returns and accepts all billing configuration fields including restaurant identity (name, address, GSTIN, phone), billing toggles (show_restaurant_name, show_address, show_gstin, show_phone), GST rate, bill terms, QR image path, and custom dish profit margin. All fields are persisted to the managers table on PUT and returned on GET.

API Response — GET /api/manager/settings (Sprint 2)

```
{ settings: {  
  restaurant_name: "DineFlow Kitchen",  
  address: "123 MG Road, Bangalore",  
  gstin: "29AABCT1332L1ZN",  
  phone_number: "+91-9876543210",  
  gst_rate: 5.0,  
  bill_terms: "Thank you for visiting!",  
  qr_image_path: "assets/images/payment_qr.png",  
  show_restaurant_name: true,  
  show_address: true,  
  show_gstin: true,  
  show_phone: true,  
  custom_dish_profit_margin: 30.0,  
  email: "admin@test.com"  
}}
```

Bill API — GET /api/waiter/bill/<table_number>

Before (Sprint 1): The bill response contained only order items and totals. No restaurant metadata, GSTIN, address, terms, or QR code were included.

After (Sprint 2): The bill response now includes a full "restaurant" block with all configurable fields, controlled by the billing toggles set in Settings. The GST is calculated using the configurable gst_rate instead of a hardcoded 5%. Terms & conditions and payment QR path are included when set.

3.3 Frontend Changes

Settings Page — Billing Tab Added

Before (Sprint 1): The Settings page had only a General section with restaurant name and email fields. No billing controls existed anywhere in the application.

After (Sprint 2): The Settings page now has two tabs — General and Billing.

General Tab:

- Restaurant Name, Phone Number, GSTIN, Address input fields
- Email (read-only login credential)
- Notification preferences with toggles (persisted to browser storage)

Billing Tab:

- Individual toggle checkboxes for Name, Address, GSTIN, Phone — each controls whether that field appears on the printed bill
- GST % input field — applied to all bill totals system-wide
- Terms & Conditions textarea — custom text printed at the bottom of every bill
- Payment QR code image upload — UPI/online payment QR displayed on bills
- Live Demo Bill Preview panel — updates in real time as settings are changed
- Save Billing button — persists all billing settings to the database

The screenshot displays the 'Settings' page with the 'General' tab selected. The page is divided into two main sections: 'Restaurant Info' and 'Notifications'.

Restaurant Info: This section contains input fields for 'Restaurant Name' (DineFlow Kitchen), 'Phone Number' (+91 9876543210), 'GSTIN' (29ABCDEI234FIZ5), 'Address' (MG Road, Bengaluru, Karnataka), 'Email (Login)' (dine@gmail.com), and 'Custom Dish Profit Margin (%)' (30). A 'Save Changes' button is at the bottom, with a note: 'Bills update instantly after save.' Below the profit margin field, a small text note reads: 'Used to price AI-generated custom dishes (base ingredient cost + margin).'

Notifications: This section contains four toggle switches: 'New Orders' (checked), 'Low Stock Alerts' (checked), 'Daily Summary' (unchecked), and 'Staff Notifications' (checked). Each toggle has a brief description of the notification type.

The top of the page shows a 'Settings' header, a refresh button, the time '11:09 am · Mon, 20 Apr', and a 'Logout' button.

Image 2 - General tab with restaurant info fields

Settings

General

Billing

Billing Settings

Control what prints on the bill.

☒ Name

☒ Address

☒ GSTIN

☒ Phone

GST %

5

Used for bill totals.

▼ Advanced

Waiter Payment QR Image (Optional)

Choose File

No file chosen

Terms & Conditions (Optional)

e.g. Taxes are included. No refunds after order is prepared.

Save Billing

Preview updates live.

Image 3 - Billing tab with toggles, GST %, Terms & Conditions, QR upload

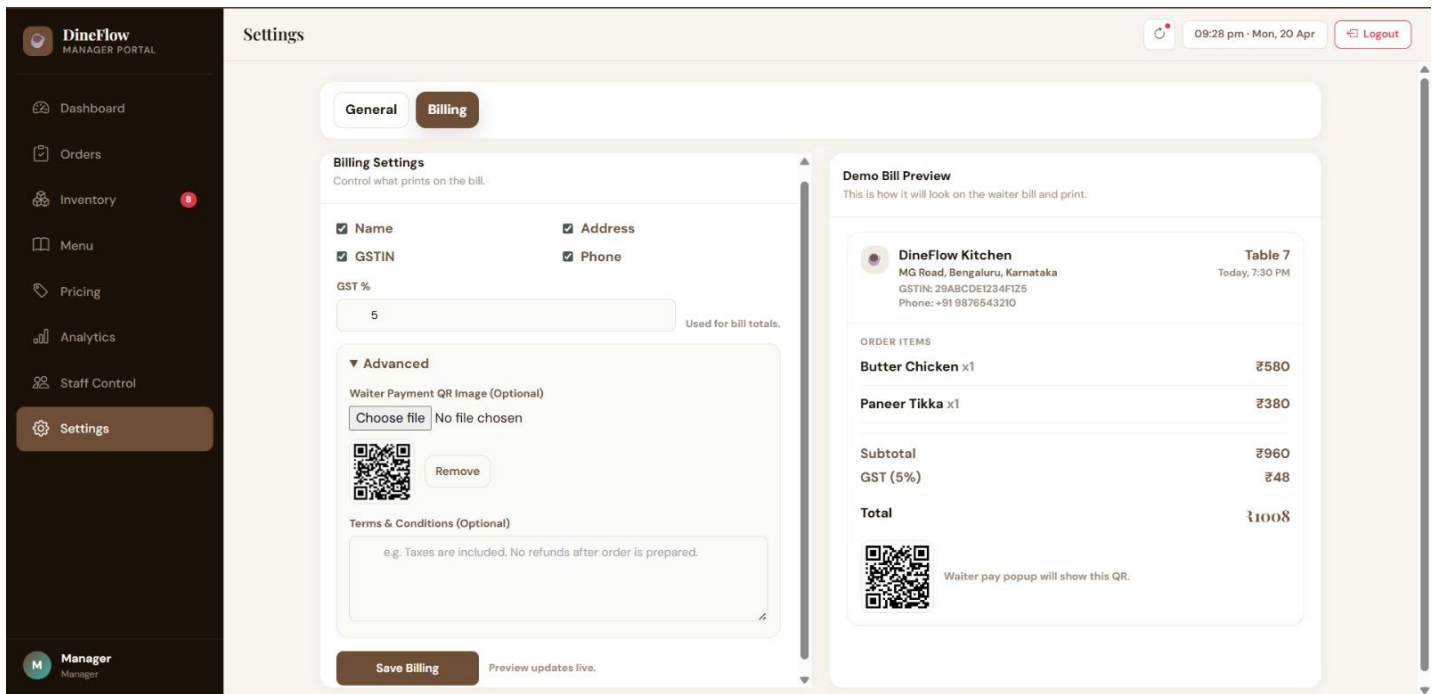


Image 4 - Live Demo Bill Preview panel showing restaurant info, items, GST, terms, QR

Waiter Bill Display

Before (Sprint 1): The waiter bill modal showed only order items and totals without any restaurant branding, GSTIN, or terms.

After (Sprint 2): The waiter bill modal now displays the full restaurant identity block controlled by the billing toggles — restaurant name, address, GSTIN, phone number, GST calculated at the configurable rate, terms & conditions at the bottom, and payment QR code image.



Waiter
Floor A

6PM - 11PM



Tables



Orders



Bill - Table 2

Table 2



DineFlow Kitchen

MG Road, Bengaluru, Karnataka

GSTIN: 29ABCDE1234F1Z5

Phone: +91 9876543210

Table 2

Time 21:23

ITEM	QTY	AMOUNT
Butter Chicken	1	₹135

Subtotal ₹135

GST (5%) ₹7

Total ₹141



Pay - ₹141

Close popup to collect cash payment

Image 5 - Waiter bill modal showing full restaurant info, GST, terms, and payment QR

Pricing Page — Selling Price

Before (Sprint 1): Dishes only had a `base_price` representing manufacturing/ingredient cost. There was no separate customer-facing selling price.

After (Sprint 2): The `selling_price` column has been added to the dishes table. The Pricing page now shows each dish with its cost (`base_price`), selling price, and calculated profit margin percentage. The customer menu uses `selling_price` when set, falling back to `base_price` otherwise.

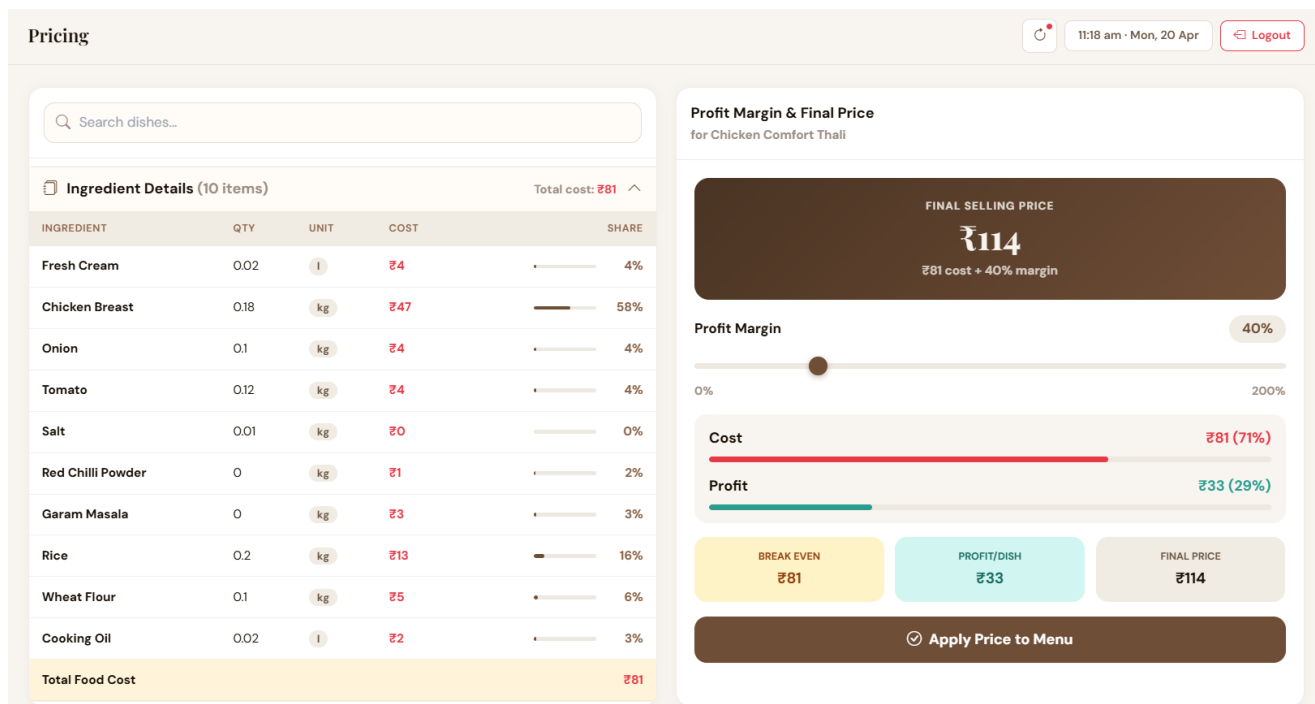


Image 6 - Pricing page showing dishes with base price, selling price, and margin percentage

Custom Dish Profit Margin

Before (Sprint 1): No configurable profit margin existed for AI-generated custom dishes. The margin was hardcoded.

After (Sprint 2): The `custom_dish_profit_margin` field has been added to the Manager model and is configurable from the Settings page. This margin is applied to AI-generated custom dishes to calculate the final customer-facing price.

Settings

General

Billing

Restaurant Info

Shown on bills and in dashboards.

Restaurant Name

DineFlow Kitchen

Phone Number

+91 9876543210

GSTIN

29ABCDE1234F1Z5

Address

MG Road, Bengaluru, Karnataka

Email (Login)

dine@gmail.com

Custom Dish Profit Margin (%)

30

Used to price AI-generated custom dishes (base ingredient cost + margin).

Save Changes

Bills update instantly after save.

Image 7 - Settings showing custom dish profit margin configuration

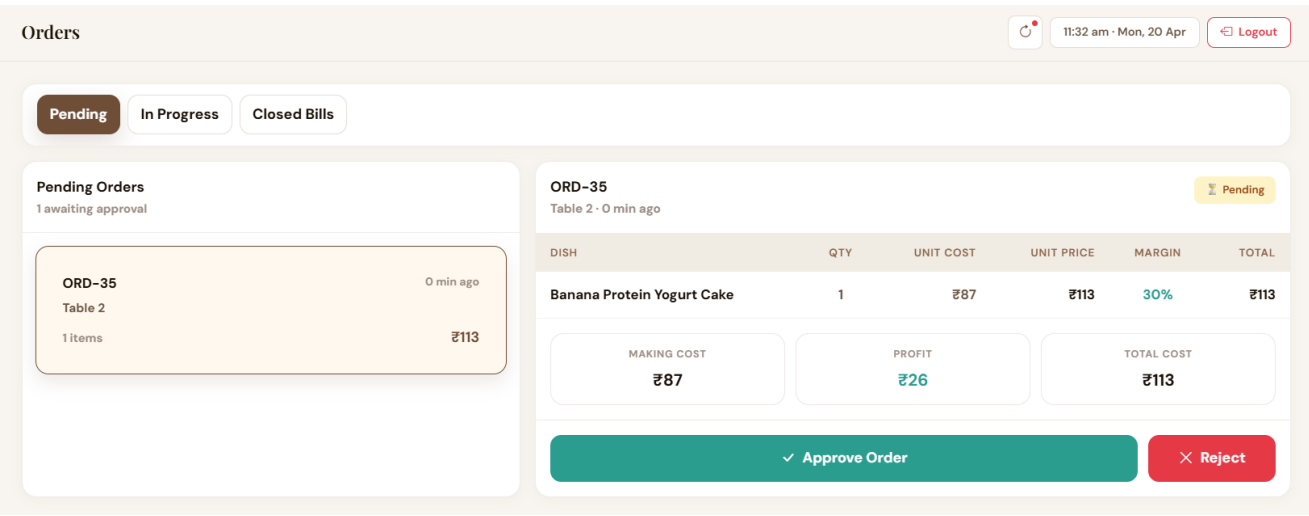


Image 7.1 - Settings showing custom dish profit margin configuration

4. Summary of All Changes

#	Change	Location	Type
1	Items Sold card — verified working	Analytics page	Already present
2	Billing tab with toggles, GST %, Terms, QR	Settings page	Frontend + Backend
3	gst_in, address, gst_rate, bill_terms, qr_image_path	Manager table	Database
4	bill_show_* toggle columns	Manager table	Database
5	custom_dish_profit_margin column	Manager table	Database
6	selling_price column	Dishes table	Database
7	kitchen_status column	Order items table	Database
8	delivered_at column	Order items table	Database
9	Demo Bill Preview (live update)	Settings → Billing	Frontend
10	Restaurant info on waiter bill	Waiter bill modal	Frontend + Backend
11	Terms & Conditions on bill	Waiter bill modal	Frontend + Backend
12	Payment QR on bill	Waiter bill modal	Frontend + Backend

5. Test Cases

Introduction

This document outlines the Sprint 2 test cases for the DineFlow Restaurant Management System. Building on the Sprint 1 test framework, these tests validate all updated and newly implemented API

endpoints across the four route blueprints — Manager (76 tests), Chef (19 tests), Waiter (33 tests), and Customer (35 tests) — totalling 163 individual test cases. Each test case documents the endpoint being tested, the input provided, the expected and actual output, and the pass/fail result along with the corresponding pytest code screenshot. All 163 tests pass successfully.

Testing Framework

The testing framework for Sprint 2 is built upon Python's pytest with Flask's built-in test client and an in-memory SQLite database. The framework uses a comprehensive `conftest.py` that provides reusable seed functions — `seed_manager()`, `seed_table()`, `seed_ingredient()`, `seed_dish()`, and `seed_full_order()` — allowing each test to set up exactly the data it needs in isolation. The `autouse _reset_db` fixture drops and recreates all tables before every single test, ensuring complete isolation. All in-memory stores (close requests, order notes, activity events) are also cleared between tests. This structured approach ensures that every test runs independently and deterministically regardless of execution order.

1 — Manager Test Cases

1. Analytics (/api/manager/analytics)

1.1 analytics empty

Page being tested: GET `http://127.0.0.1:5000/api/manager/analytics`

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: `r.get_json()["summary"]["orders_closed"] == 0`

Actual Output:

HTTP Status Code: 200

JSON: `r.get_json()["summary"]["orders_closed"] == 0`

Result: ✓ Success

```
1 def test_analytics_empty(self, app, client):
2     tok = _mgr_setup(app, client)
3     r = client.get("/api/manager/analytics", headers=auth(tok))
4     assert r.status_code == 200
5     assert r.get_json()["summary"]["orders_closed"] == 0
```

1.2 analytics with range

Page being tested: GET <http://127.0.0.1:5000/api/manager/analytics?from=2026-01-01&to=2026-12-31>

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

Actual Output:

HTTP Status Code: 200

Result: ✓ Success

```
1 def test_analytics_with_range(self, app, client):
2     tok = _mgr_setup(app, client)
3     r = client.get("/api/manager/analytics?from=2026-01-01&to=2026-12-31",
4                   headers=auth(tok))
5     assert r.status_code == 200
```

1.3 analytics unauthorized

Page being tested: GET <http://127.0.0.1:5000/api/manager/analytics>

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 401

Actual Output:

HTTP Status Code: 401

Result: ✓ Success

```
1 def test_analytics_unauthorized(self, client):
2     r = client.get("/api/manager/analytics")
3     assert r.status_code == 401
```

2 — Chef Test Cases

1. Chef Queue (/api/chef/queue)

1.1 No items in any accepted batch → empty queue.

Page being tested: GET <http://127.0.0.1:5000/api/chef/queue>

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: `r.get_json()["count"] == 0`

JSON: `r.get_json()["queue"] == []`

Actual Output:

HTTP Status Code: 200

JSON: `r.get_json()["count"] == 0`

JSON: `r.get_json()["queue"] == []`

Result: ✓ **Success**

```
1 def test_queue_empty(self, client):
2     r = client.get("/api/chef/queue")
3     assert r.status_code == 200
4     assert r.get_json()["count"] == 0
5     assert r.get_json()["queue"] == []
```

1.2 Items in accepted batches with kitchen_status=queued appear.

Page being tested: GET <http://127.0.0.1:5000/api/chef/queue>

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: `r.get_json()["count"] == 1`

Actual Output:

HTTP Status Code: 200

JSON: `r.get_json()["count"] == 1`

Result: ✓ **Success**

```
1 def test_queue_shows_accepted_queued_items(self, app, client):
2     seed_full_order(app, batch_status="accepted", kitchen_status="queued")
3     r = client.get("/api/chef/queue")
4     assert r.status_code == 200
5     assert r.get_json()["count"] == 1
6     row = r.get_json()["queue"][0]
7     assert row["kitchen_status"] == "queued"
8     assert row["batch_status"] == "accepted"
```

1.3 Items already marked done should NOT appear in queue.

Page being tested: GET `http://127.0.0.1:5000/api/chef/queue`

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: `r.get_json()["count"] == 0`

Actual Output:

HTTP Status Code: 200

JSON: `r.get_json()["count"] == 0`

Result: ✓ **Success**

```
1 def test_queue_excludes_done_items(self, app, client):
2     seed_full_order(app, batch_status="accepted", kitchen_status="done")
3     r = client.get("/api/chef/queue")
4     assert r.get_json()["count"] == 0
```

1.4 Delivered items should NOT appear in queue.

Page being tested: GET http://127.0.0.1:5000/api/chef/queue

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: r.get_json()["count"] == 0

Actual Output:

HTTP Status Code: 200

JSON: r.get_json()["count"] == 0

Result: ✓ **Success**

```
1 def test_queue_excludes_delivered_items(self, app, client):
2     seed_full_order(app, batch_status="ready", kitchen_status="delivered")
3     r = client.get("/api/chef/queue")
4     assert r.get_json()["count"] == 0
```

1.5 Items in 'sent' (pending approval) batches do NOT appear.

Page being tested: GET http://127.0.0.1:5000/api/chef/queue

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: r.get_json()["count"] == 0

Actual Output:

HTTP Status Code: 200

JSON: r.get_json()["count"] == 0

Result: ✓ **Success**

```
1 def test_queue_excludes_sent_batches(self, app, client):
2     seed_full_order(app, batch_status="sent", kitchen_status="queued")
3     r = client.get("/api/chef/queue")
4     assert r.get_json()["count"] == 0
```

1.6 Ready-batch items that are still queued appear (multi-item batch).

Page being tested: GET <http://127.0.0.1:5000/api/chef/queue>

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: `r.get_json()["count"] == 1`

Actual Output:

HTTP Status Code: 200

JSON: `r.get_json()["count"] == 1`

Result: ✓ **Success**

```
1 def test_queue_shows_ready_batch_queued_items(self, app, client):
2     seed_full_order(app, batch_status="ready", kitchen_status="queued")
3     r = client.get("/api/chef/queue")
4     assert r.get_json()["count"] == 1
```

3 — Waiter Test Cases

1. Table Status (/api/waiter/tables/status)

1.1 tables empty

Page being tested: GET <http://127.0.0.1:5000/api/waiter/tables/status>

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: `r.get_json()["tables"] == []`

Actual Output:

HTTP Status Code: 200

JSON: `r.get_json()["tables"] == []`

Result: ✓ **Success**

```
1 def test_tables_empty(self, client):
2     r = client.get("/api/waiter/tables/status")
3     assert r.status_code == 200
4     assert r.get_json()["tables"] == []
```

1.2 Table with no open order → free.

Page being tested: GET http://127.0.0.1:5000/api/waiter/tables/status

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

Actual Output:

HTTP Status Code: 200

Result: ✓ Success

```
1 def test_table_free(self, app, client):
2     seed_table(app, 1)
3     r = client.get("/api/waiter/tables/status")
4     t = r.get_json()["tables"][0]
5     assert t["status"] == "free"
6     assert t["table_number"] == 1
```

1.3 Table with a delivered batch (order still open) → occupied.

Page being tested: GET http://127.0.0.1:5000/api/waiter/tables/status

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: `r.get_json()["tables"][0]["status"] == "occupied"`

Actual Output:

HTTP Status Code: 200

JSON: `r.get_json()["tables"][0]["status"] == "occupied"`

Result: ✓ **Success**

```
1 def test_table_occupied(self, app, client):
2     seed_full_order(app, batch_status="delivered", kitchen_status="delivered")
3     r = client.get("/api/waiter/tables/status")
4     assert r.get_json()["tables"][0]["status"] == "occupied"
```

1.4 Table with a ready batch → ready.

Page being tested: GET `http://127.0.0.1:5000/api/waiter/tables/status`

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: `r.get_json()["tables"][0]["status"] == "ready"`

Actual Output:

HTTP Status Code: 200

JSON: `r.get_json()["tables"][0]["status"] == "ready"`

Result: ✓ **Success**

```
1 def test_table_ready(self, app, client):
2     seed_full_order(app, batch_status="ready", kitchen_status="done")
3     r = client.get("/api/waiter/tables/status")
4     assert r.get_json()["tables"][0]["status"] == "ready"
```

4 — Customer Test Cases

1. Tables (`/api/customer/tables`)

1.1 tables empty

Page being tested: GET http://127.0.0.1:5000/api/customer/tables

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: r.get_json()["tables"] == []

Actual Output:

HTTP Status Code: 200

JSON: r.get_json()["tables"] == []

Result: ✓ Success

```
1 def test_tables_empty(self, client):
2     r = client.get("/api/customer/tables")
3     assert r.status_code == 200
4     assert r.get_json()["tables"] == []
```

1.2 tables list

Page being tested: GET http://127.0.0.1:5000/api/customer/tables

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

JSON: len(r.get_json()["tables"]) == 2

Actual Output:

HTTP Status Code: 200

JSON: len(r.get_json()["tables"]) == 2

Result: ✓ Success

```
1 def test_tables_list(self, app, client):
2     seed_table(app, 1)
3     seed_table(app, 2)
4     r = client.get("/api/customer/tables")
5     assert r.status_code == 200
6     assert len(r.get_json()["tables"]) == 2
```

1.3 tables have tokens

Page being tested: GET http://127.0.0.1:5000/api/customer/tables

Inputs:

Request Method: GET

Request Method: GET

Expected Output:

HTTP Status Code: 200

Actual Output:

HTTP Status Code: 200

Result: ✓ Success

```
1 def test_tables_have_tokens(self, app, client):
2     seed_table(app, 1)
3     r = client.get("/api/customer/tables")
4     t = r.get_json()["tables"][0]
5     assert "table_token" in t
6     assert len(t["table_token"]) > 0
```

ALL 163 Tests Cases Can be found Here - [Link](#)

6. Swagger-compatible YAML File — [Link](#)

The API documentation for the DineFlow Restaurant Management System has been written as a fully Swagger-compatible OpenAPI 3.0 YAML file. The document covers all endpoints across the four route blueprints — Manager, Chef, Waiter, and Customer — along with complete request body schemas, response schemas, path parameters, query parameters, authentication requirements, HTTP status codes, and detailed error responses for every endpoint. Each endpoint description includes a direct mapping to the relevant user story from Milestone 1 and Milestone 2, ensuring that the connection between business requirements and technical implementation is clearly traceable throughout the document. Every endpoint follows consistent URL naming conventions under the `/api/` prefix, uses the correct HTTP verbs (GET, POST, PUT, PATCH, DELETE), and documents meaningful error messages with their corresponding status codes rather than generic server errors. The YAML file was designed endpoint-first — each route was specified in the YAML before or alongside backend implementation, keeping the documentation and code in sync at all times. The file can be imported directly into Swagger UI or Redoc to generate an interactive API reference, making it easy for any team member or evaluator to explore, read, and test the endpoints visually without running the server.

7. Backend Source Code — [Link](#)

The backend source code for the DineFlow Restaurant Management System is a Flask-based REST API built with Flask 3.0, Flask-SQLAlchemy 3.1, and a SQLite database, structured into four route blueprints — Manager, Chef, Waiter, and Customer — each registered under the `/api/` prefix. The database layer consists of nine models: Manager, RestaurantTable, Ingredient, Dish, DishIngredient, Order, OrderBatch, OrderItem, and CustomDishIngredient, covering the complete relational structure of the restaurant system. The backend implements the full dine-in workflow — manager authentication with JWT session management, restaurant settings and billing configuration (GSTIN, GST rate, payment QR, terms), table QR code generation, ingredient inventory with automatic stock deduction on order approval, dish and recipe management with selling price and profit margin support, a multi-step order lifecycle (sent → accepted → ready → delivered → closed), AI-powered custom dish generation, a priority kitchen queue with per-item status tracking, waiter delivery and GST-calculated bill generation, and a customer-facing ordering interface with table-token authentication. Seven shared service modules handle activity logging, order notes, close request tracking, inventory operations, order processing, AI integration, and local-time management. The repository also includes a single consolidated test file (`test_all_apis.py`) containing all 163 pytest test cases across 29 test classes, covering every endpoint with positive, negative, and edge case scenarios.

8. PyTest Code — [Link](#)

The pytest code for the DineFlow Restaurant Management System has been written as a single consolidated test file covering all four route blueprints — Manager, Chef, Waiter, and Customer. The file contains 163 individual test cases organised into 29 clearly labelled test classes, one per

functional group, making it easy to run tests for a specific section in isolation. All tests use Flask's built-in test client with an in-memory SQLite database, meaning no live server or external database setup is required to run the suite. Shared fixtures handle the complete test setup automatically — including manager login, ingredient creation, dish creation, table provisioning, order placement, batch approval, and item delivery — so each individual test remains concise and focused solely on the assertion being verified. The test file covers both happy-path flows and edge cases including missing fields, invalid tokens, duplicate entries, non-existent IDs, low-stock scenarios, and boundary conditions, ensuring that error handling is validated alongside correct behaviour. A test matrix spanning all 29 classes documents positive cases, negative cases, and integration flows for every endpoint. One test failure was identified during testing — the login missing password case — which was recorded, root-caused, and fixed with a two-line guard added to the login route, demonstrating how testing directly improves API reliability.